

Programming UNIX

Configured



Module 10:

Multithreading

Overview

➤ In this module you will learn about:

- **Threads**
- **Creating Threads**
- **Terminating Threads**
- **Thread Synchronization**
- **Synchronizing Pthreads**
- **Coordinating Pthreads**

Lesson: Threads

➤ This lesson describes:

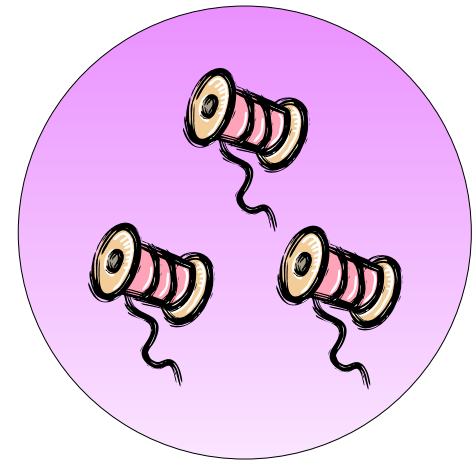
- **Multithreading**
- **Pthreads**
- **Benefits of using Threads**
- **Using Treads in Applications**

Multithreading

➔ If the main thread is blocked or waiting you can create another threads in the same process

➔ All threads in a process

- Share open files
- Share the process image
- Have own **registers** and **stack**



➔ By **context switching** the OS

- A** **Processes** - handles info about process data (~ m sesc)
- B** **Threads** - handles registers and stacks (~ μ secs)

Threads

➡ A **thread** has information necessary to represent an execution context within a process:

- A thread ID that identifies the thread within a process
- A set of register values
- A stack
- A scheduling priority and policy
- A signal mask
- An errno variable

➡ Pthreads - an optional feature in POSIX.1-2001

- The **_POSIX_THREADS** macro tests for POSIX threads

Benefits of Threads

➡ The major benefits of threads:

A Improving Performance on:

- **Single Processes** – threads run independently
- **Multiprocessors** - threads can run simultaneously

B Natural Modeling

- Making easier the model how humans work and interact

C Handling Asynchronous Events

- E.g. Multiple independent communications

D More Responsive GUIs

- Modern GUI frameworks have an event dispatch thread

Using Threads



Threads can be useful for:

- **Background printing and saving**
- **Background downloading**
- **GUIs**
- **Redrawing in graphics programs**
- **Concurrent servers**
 - Connection and communication with multiple clients
- **Parallel processing**
 - Dividing a computational program into subtasks that can run on separate threads
- ...

Lesson: Creating and Terminating

➔ This lesson describes:

- **Creating Threads**
- **Achieving Parallelism**
- **Terminating Threads**
- **Waiting For Terminations**
- **Cancelling Threads**
- **Thread Cleanup Handlers**

pthread_create

➡ **pthread_create** – creates a new thread

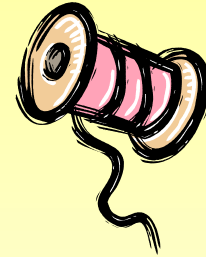
```
#include <pthread.h>
```

```
int pthread_create (pthread_t *restrict tidp,  
                    const pthread_attr_t *restrict attr,  
                    void *(*start_rtn)(void),  
                    void *restrict arg);
```

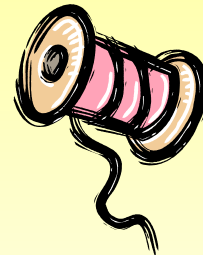
- tidp – a pointer to a thread id (tid)
- attr – thread attributes – NULL for default
- start-rtn – a function pointer to function to be executed
- arg – argument passed to the start routine

Creating Threads

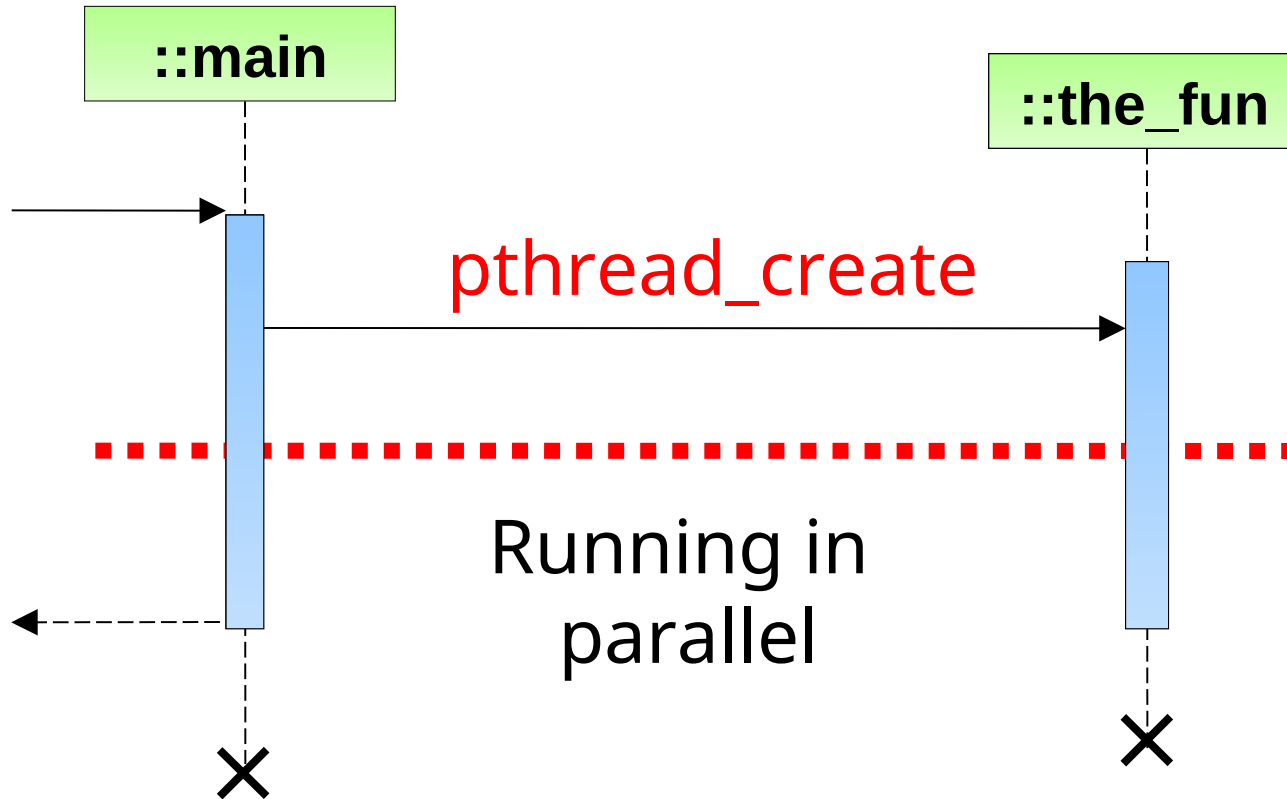
```
main () {  
    pthread_t tid;  
    int err;  
    err = pthread_create (&tid, NULL, thr_fn, NULL);  
}
```



```
void *thr_fn (void *arg) {  
    Thread Code  
    return ((void *)0);  
}
```



Achieving Parallelism



Terminating Threads

➤ If any thread calls `exit`, `_Exit`, or `_exit` the entire process terminates.

➤ You can terminate a thread:

- 1 The thread returns from the start routine
 - The return value is the thread's exit code
- 2 The thread is canceled by another thread in the same process
- 3 The thread can call `pthread_exit`.

```
#include <pthread.h>
```

```
void pthread_exit (void *rval_ptr);
```

Waiting for Termination

➡ A thread can block waiting for another thread to terminate calling **pthread_join**

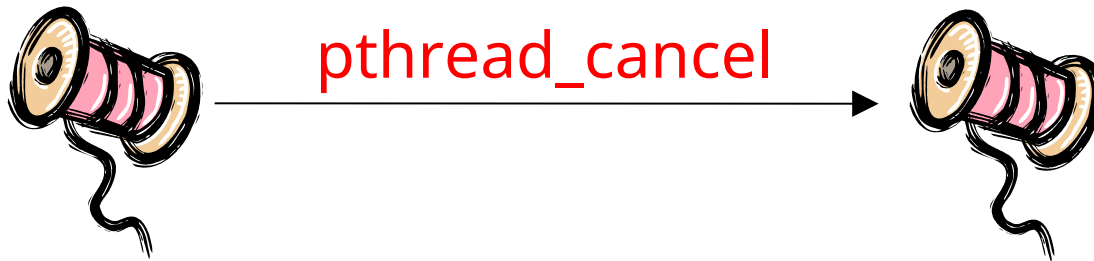
```
#include <pthread.h>
```

```
int pthread_join (pthread_t thread, void **rval_ptr);
```

- **rval_ptr** – contains:
 - The return code - if the thread simply returned from its start routine
 - PTHREAD_CANCELED - If the thread was canceled

Canceling Threads

➡ One thread can request that another in the same process to be canceled by calling **pthread_cancel**



```
#include <pthread.h>
```

```
int pthread_cancel (pthread_t tid);
```

- This is only the notification to a thread
 - It can **ignore** the cancellation request

Setting Cleanup Handlers

- A thread can set functions to be called when it exits
 - Thread cleanup handlers – recorded on the stack
- You can set multiple cleanup handlers for a thread
 - They executed in the reverse order

```
#include <pthread.h>
```

```
void pthread_cleanup_push (void (*rtn)(void *), void *arg);
```

```
void pthread_cleanup_pop (int execute);
```

- **rtn** – a pointer to a clean up function

Executing Cleanup Handlers

➤ A cleanup function is called when the thread performs one of the following actions:

- 1 Executes `pthread_exit`
- 2 Responds to a cancellation request
- 3 Executes `pthread_cleanup_pop` with a nonzero execute argument
 - If the execute argument is set to zero, the cleanup function is not called.
 - `pthread_cleanup_pop` always removes the cleanup handler

Practice: Creating and Terminating



□ In this practice you will be shown how to create and terminate threads.

Lesson: Thread Synchronization

➤ This lesson describes:

- **Critical Sections**
- **Race Condition**
- **Locking Mechanisms**
- **Mutual Exclusion**

Critical Sections

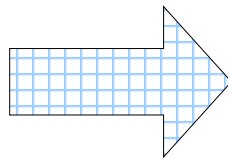
➤ Problem using threads may arise (**race condition**) in situations when

- Multiple threads access shared data *and*
- The final result depends who run precisely when

➤ For instance, a C **count ++** statement is not *atomic* and can be a subject of race condition

- It is translated into three assembler instructions and the scheduler can switch the thread inside the **critical section**

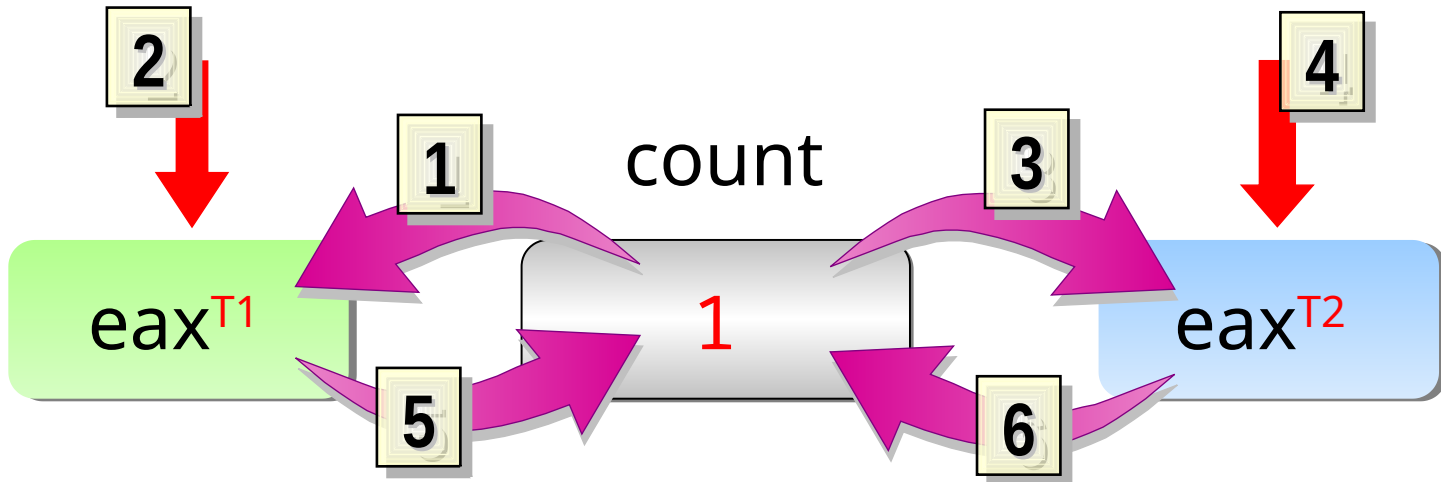
```
count++;
```



```
mov eax, [count]
inc eax
mov [count], eax
```

Race Condition

- ➡ In this example, variable **count** is initialized to **1**.
- The correct result after both threads are executed is **3**
 - The following interleaving yields **2** what is an *incorrect result*



➡ **Solution:** Synchronize concurrent accesses inside critical sections

Protecting Critical Code

➤ To protect critical code you must provide **mutual exclusion** using

- synchronization techniques

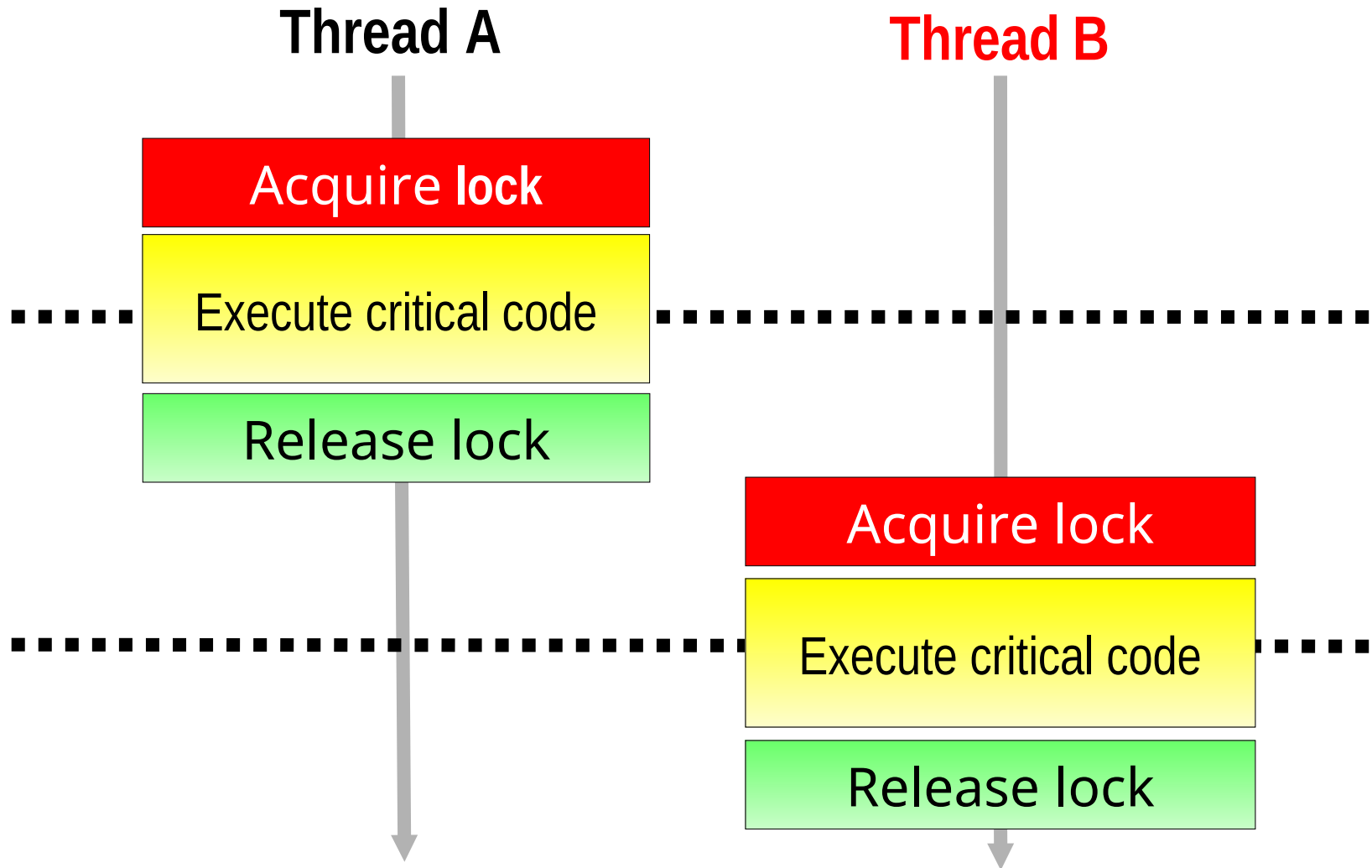
➤ When synchronizing threads **must**

- 1 Obtain a **lock**
- 2 Execute the critical code
- 3 Release the **lock**



➤ All threads should apply the synchronization techniques

Mutual Exclusion



Lesson: Synchronizing Pthreads

➤ This lesson describes:

- **Mutexes**
- **Locking**
- **Unlocking**
- **Deadlocking**

Using Mutexes

➡ A **mutex** is a lock that is:

- Locked (set) before accessing a shared resource
- Unlocked (released) when we're done

➡ **pthread_mutex_t** - represents a mutex variable

- A mutex variable must be initialized

```
#include <pthread.h>
```

```
int pthread_mutex_init (pthread_mutex_t *restrict mutex,  
                        const pthread_mutexattr_t *restrict attr);
```

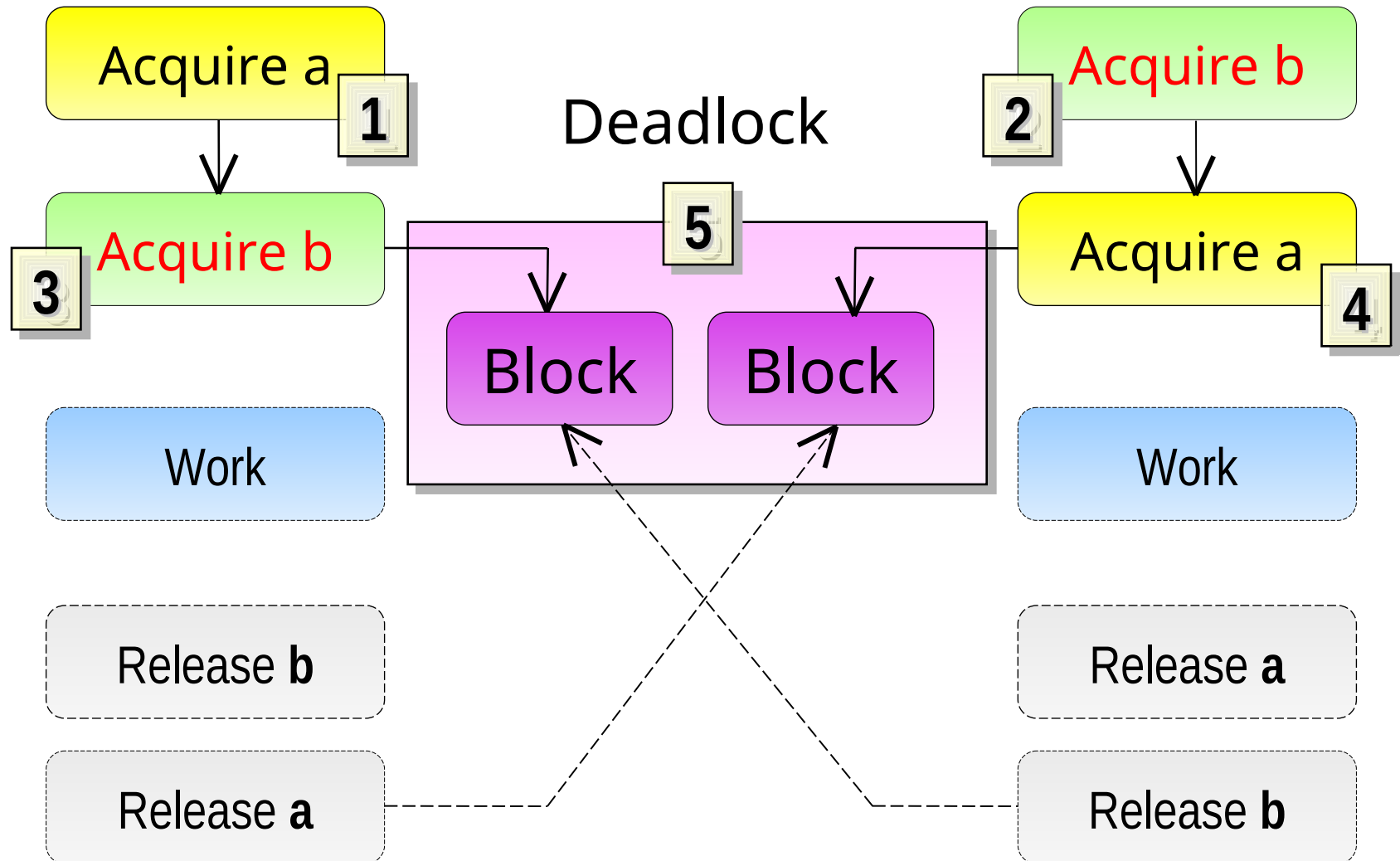
```
int pthread_mutex_destroy (pthread_mutex_t *mutex);
```


Locking and Unlocking

➡ The following routines are used:

- **pthread_mutex_lock** - locks a mutex
 - If the mutex is already locked, the calling thread will block until the mutex is unlocked
- **pthread_mutex_trylock** - locks a mutex conditionally:
 - If the mutex is unlocked it will lock the mutex without blocking and return **0**.
 - Otherwise, it will fail and return **EBUSY**
- **pthread_mutex_unlock** - unlocks a mutex

Deadlock



Practice: Synchronizing Pthreads

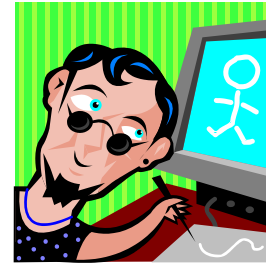


□ In this practice you will be shown how to synchronize access to critical code regions using synchronization mechanisms.

Lesson: Coordinating Pthreads

➡ This lesson describes:

- **Busy Waiting**
- **A Circular Buffer**
- **Thread Coordination**
- **Thread Coordination Patterns**



D. Lea

Busy Waiting

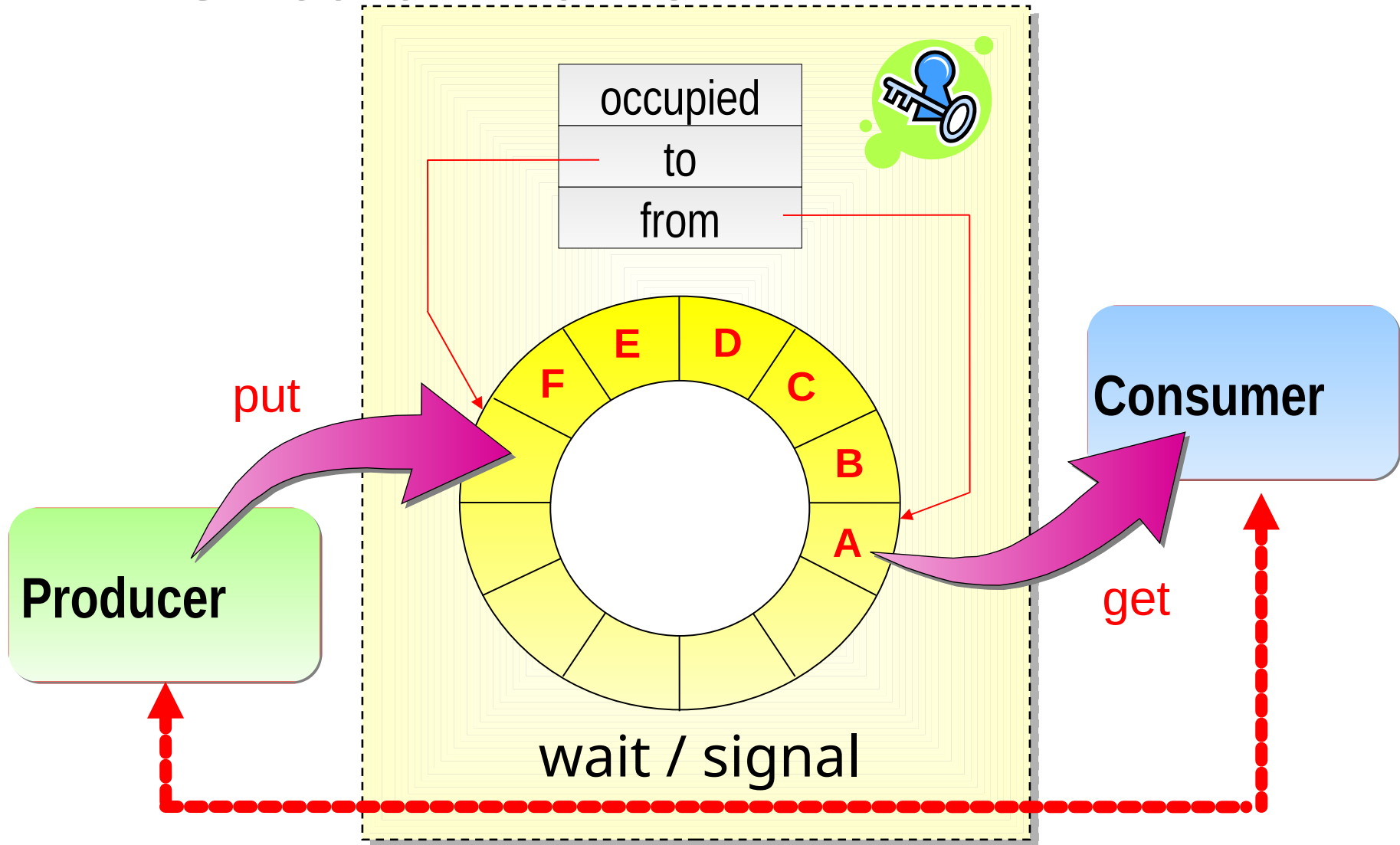
➤ Threads must coordinate to avoid **busy waiting**

- **Locking mechanisms** are used to prevent threads to execute critical code at the same time

➤ Threads can coordinate using

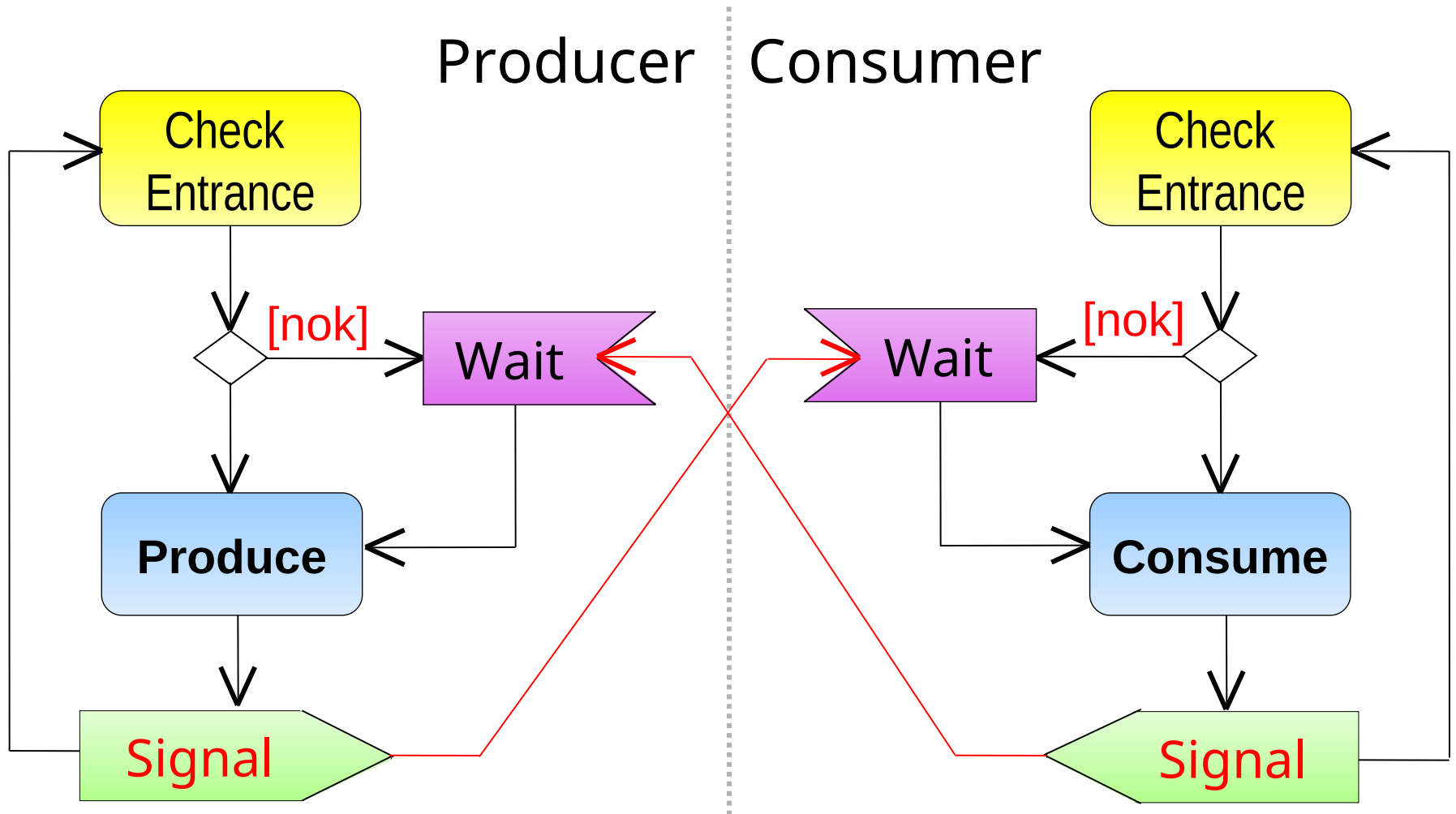
- **pthread_cond_wait** – indivisibly releases the lock and puts the thread to the waiting state
- **pthread_cond_signal** - wakes up one thread
- **pthread_cond_broadcast** - wakes up all threads waiting on a condition

A Circular Buffer



Thread Coordination

Producer Consumer



Coordination Patterns

➡ The pattern for waiting

```
pthread_mutex_lock (&lock);  
while (something is true)  
    pthread_cond_wait (&cond, &lock);  
Can proceed
```

➡ The pattern for signaling

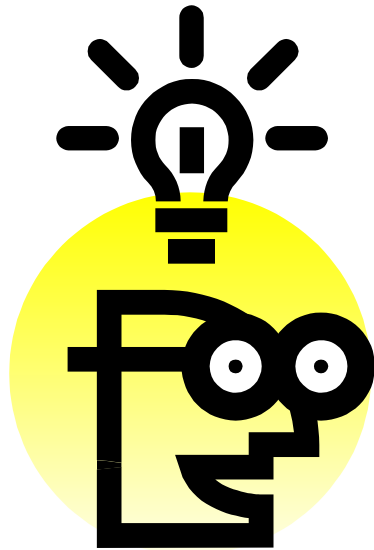
```
pthread_mutex_unlock (&lock);  
pthread_cond_signal (&cond);
```


Practice: Coordinating Pthreads



□ In this practice you will be shown how to coordinate threads when accessing shared resources.

Review



- How can you pass multiple parameters to a thread?
- What is the difference between functions `exit` and `pthread_exit`?
- Can a mutex differentiate between readers and writers?
- What is the difference between thread synchronization and coordination?